

Salient Object Detection Using a CNN Encoder-Decoder

MSRA10K Dataset Report

Project #3: End-to-End ML/DL Project

Python, PyTorch, Google Colab

Prepared by: Milot Hyseni

COHORT V

Contents

1. Introduction and Project Objective.....	3
1.1 Main Objectives.....	3
2.1 Data Augmentation	3
3.1 Source Code Organization.....	4
4. Model Design.....	5
4.1 Baseline CNN Encoder-Decoder	5
4.2 Improved CNN Model.....	5
4.3 Output Shape Verification	5
5.1 Loss Function.....	5
5.2 Training Configuration	6
5.3 Key Training Results	6
7.1 Result Interpretation	7
7.2 Final Model Selection	7
8.1 Notebook Demo	8
9. Discussion, Limitations, and Future Work.....	8
9.1 Discussion.....	8
9.2 Limitations	8
9.3 Future Work.....	8
10. Conclusion.....	8
10.1 Final Deliverables.....	8

1. Introduction and Project Objective

Salient Object Detection (SOD) is a computer vision task focused on identifying the most visually important region in an image. Unlike traditional object detection, the goal is not to classify every object, but to produce a pixel-level mask that highlights the dominant salient area.

This project implements an end-to-end SOD pipeline from scratch using PyTorch. The system includes dataset preparation, preprocessing, augmentation, a CNN encoder-decoder architecture, custom training logic, evaluation metrics, visualizations, and a notebook demo that accepts an uploaded image and produces a saliency mask with an overlay.

1.1 Main Objectives

- Load and inspect the MSRA10K saliency dataset.
- Resize and normalize images and binary masks to a consistent 128x128 format.
- Split the dataset into train, validation, and test subsets using a 70/15/15 split.
- Build a CNN encoder-decoder model from scratch without using a pretrained model.
- Train using Binary Cross-Entropy combined with IoU loss and the Adam optimizer.
- Evaluate the model using IoU, Precision, Recall, and F1-score.
- Compare a baseline model with an improved model using quantitative and qualitative evidence.
- Provide a demo notebook that uploads any image and displays the predicted mask, overlay, and inference time.

2. Dataset Preparation and Preprocessing

The MSRA10K dataset was selected because it is a standard public benchmark for saliency tasks and contains 10,000 RGB images with corresponding ground-truth saliency masks. The dataset was downloaded automatically in Google Colab using the Kaggle API when the expected folder was missing.

Item	Project Setting
Dataset	MSRA10K
Images	10,000 JPG images
Masks	10,000 PNG binary saliency masks
Input size	128 x 128
Split	70% train, 15% validation, 15% test
Batch size	16
Environment	Google Colab with CUDA GPU

All images were converted to RGB tensors with shape [3, 128, 128]. Masks were converted to grayscale tensors with shape [1, 128, 128] and thresholded to remain binary-like after transformations. Pixel values were normalized to the [0, 1] range using tensor conversion.

<pre>... Dataset exists: True Images: 10000 Masks: 10000</pre>	<pre>... Train batches: 438 Validation batches: 94 Test batches: 94</pre>
--	---

Figure 1: Dataset verification and DataLoader batch counts.

2.1 Data Augmentation

Augmentation was applied only to the training set so that validation and test results remain unbiased. Geometric transformations were applied consistently to both the image and the mask, while color jitter was applied only to the image.

- Random horizontal flip
- Small random rotation

- Random resized crop
- Brightness, contrast, and saturation variation

3. Pipeline Verification and Project Structure

Before training, batch shape checks and visualization sanity checks were used to confirm that images and masks were aligned correctly. This is important in segmentation because a small mismatch between an image and its mask would make the model learn incorrect pixel labels.

```
Image batch shape: torch.Size([16, 3, 128, 128])
Mask batch shape: torch.Size([16, 1, 128, 128])
Image min/max: 0.0 1.0
Mask min/max: 0.0 1.0
```

Figure 2: Batch tensor shape and value-range verification.

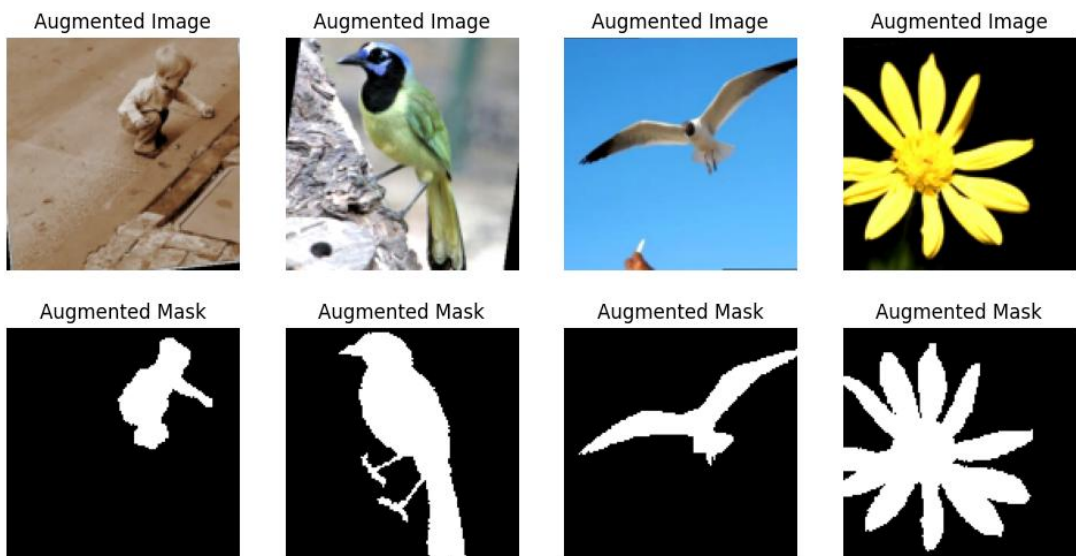


Figure 3: Augmented training image-mask pairs.

3.1 Source Code Organization

The project was organized into separate modules so each part of the ML pipeline is easy to test, reuse, and review.

File	Purpose
data_loader.py	Dataset loading, resizing, normalization, augmentation, and DataLoaders.
sod_model.py	Baseline and improved CNN encoder-decoder models.
train.py	Training/validation loop, combined loss, checkpoint saving, and best model saving.
evaluate.py	IoU, Precision, Recall, F1 metric computation and visualizations.
demo_notebook.ipynb	Colab workflow, visualizations, model comparison, and upload-image demo.
checkpoints/	Best and last model weights for baseline and improved runs.

4. Model Design

The SOD task was treated as binary image segmentation. For each input image, the model predicts one output value per pixel, where values closer to 1 represent salient pixels and values closer to 0 represent background pixels. Both models output a one-channel sigmoid mask with the same spatial size as the input.

4.1 Baseline CNN Encoder-Decoder

The baseline model uses a simple encoder-decoder structure. The encoder contains three convolutional blocks with ReLU activations and max pooling. The decoder uses transposed convolutions to upsample the feature map back to 128x128. A final 1x1 convolution followed by sigmoid activation produces the saliency mask.

Component	Baseline Design
Input	RGB image [B, 3, 128, 128]
Encoder	Conv 3->32, Conv 32->64, Conv 64->128, each followed by MaxPool
Decoder	ConvTranspose 128->64, 64->32, 32->16
Output	1-channel sigmoid mask [B, 1, 128, 128]

4.2 Improved CNN Model

The improved model keeps the same encoder-decoder idea but adds stronger representation learning. Each encoder stage uses two convolutional blocks, and the blocks include Batch Normalization and Dropout. These changes improve training stability and reduce overfitting while keeping the model fully built from scratch.

Improvement	Reason
Deeper encoder blocks	Extract richer visual features from the image.
Batch Normalization	Stabilizes activations and makes training smoother.
Dropout	Reduces overfitting by randomly dropping feature maps.
Augmentation	Improves generalization to unseen images.

4.3 Output Shape Verification

A forward-pass test was performed before training. Both the baseline and improved models produced masks with shape [B, 1, 128, 128], confirming that the decoder restored the output to the correct spatial dimensions.

5. Training Methodology

Both models were trained with PyTorch using a custom training loop. Each epoch performed a forward pass, loss calculation, backward pass, optimizer update, validation pass, and checkpoint saving. The Adam optimizer was used with a learning rate of 1e-3. As an optional bonus feature, checkpoint save and resume logic was implemented. After every epoch, the training loop saves the current epoch, model weights, optimizer state, training losses, and validation losses. If the notebook is restarted or training is interrupted, the system automatically detects the latest checkpoint and resumes training from the next epoch. Console messages are printed to confirm when checkpoints are saved and when training is resumed.

5.1 Loss Function

The training objective combined pixel-wise classification quality with mask-overlap quality. Binary Cross-Entropy helps the model learn foreground/background classification per pixel, while IoU loss encourages the predicted mask to overlap with the ground-truth saliency region.

$$\text{Total Loss} = \text{Binary Cross-Entropy} + 0.5 * (1 - \text{IoU})$$

5.2 Training Configuration

Parameter	Value
Framework	PyTorch
Optimizer	Adam
Learning rate	1e-3
Epochs	20
Batch size	16
Input size	128 x 128
Checkpointing	Best model by validation loss plus last checkpoint

5.3 Key Training Results

The long epoch logs were reduced to the most important checkpoints. The improved model reached its best validation loss at epoch 18, while the baseline best validation loss was reached at epoch 20.

Model	Epoch 1 Val Loss	Best Val Loss	Best Epoch	Epoch 20 Val Loss
Baseline	1.0188	0.7885	20	0.7885
Improved + Augmentation	0.8936	0.5201	18	0.5454

6. Evaluation Metrics and Training Curves

The trained models were evaluated on the held-out test set using thresholded predicted masks. The threshold used for binary masks was 0.5. Four standard segmentation metrics were calculated: IoU, Precision, Recall, and F1-score.

Metric	Meaning
IoU	Measures overlap between predicted and ground-truth masks. Higher is better.
Precision	Measures how many predicted salient pixels were correct.
Recall	Measures how many true salient pixels were detected.
F1-score	Harmonic mean of Precision and Recall.

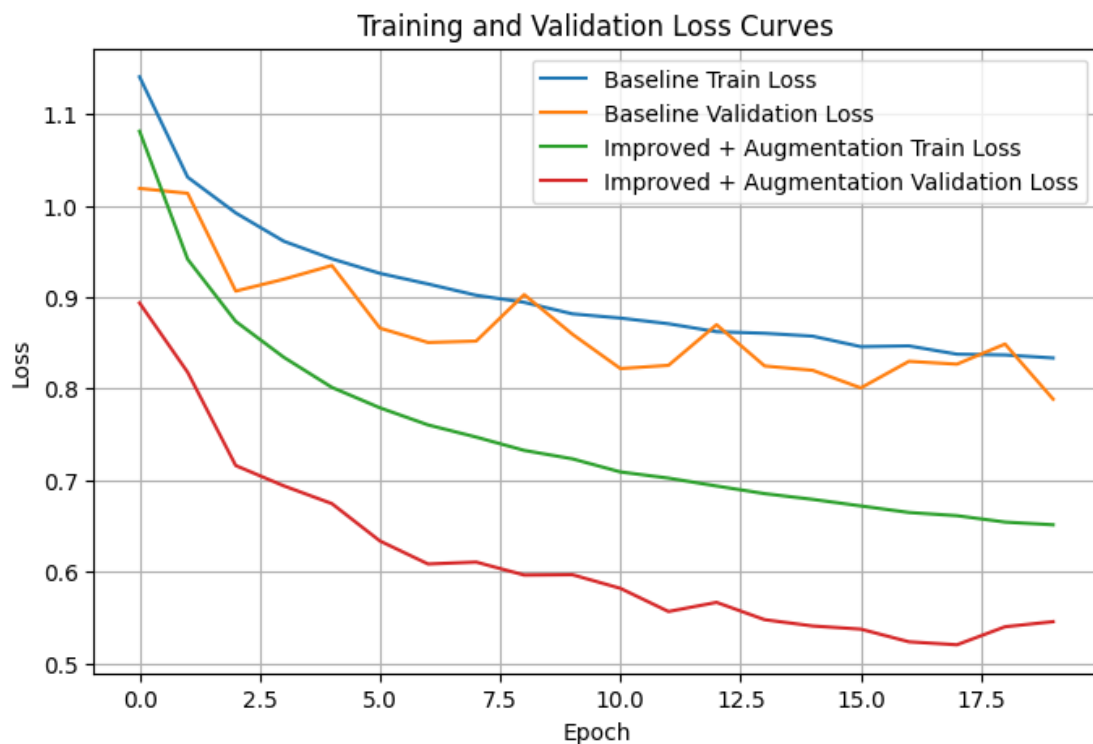


Figure 4: Training and validation loss curves for the baseline and improved models.

The improved model has consistently lower validation loss than the baseline. Its validation loss decreased quickly during the first epochs and stayed substantially below the baseline curve, which supports the quantitative improvement observed on the test set.

7. Quantitative Results

The improved model outperformed the baseline model across all evaluation metrics. This confirms that the architectural improvements and augmentation strategy increased the quality of the predicted saliency masks.

Model	IoU	Precision	Recall	F1-score
Baseline	0.6004	0.7186	0.8018	0.7247
Improved + Augmentation	0.7270	0.8068	0.8851	0.8256
Absolute Improvement	+0.1266	+0.0882	+0.0833	+0.1009

7.1 Result Interpretation

The IoU increase from 0.6004 to 0.7270 is the strongest evidence that the improved model produced masks with better spatial overlap. Precision also improved, meaning fewer background pixels were incorrectly labeled as salient. Recall improved as well, meaning the model detected more of the actual salient region. The F1-score improvement shows that the model achieved a better balance between precision and recall.

7.2 Final Model Selection

The improved model with augmentation was selected as the final model because it achieved the best overall score and produced clearer qualitative masks. The baseline model remains useful as a comparison point because it demonstrates the value of the improvements.

8. Qualitative Results and Demo

Qualitative visualization was used to inspect whether the predicted masks identify the same salient regions as the ground-truth masks. Each visualization shows the input image, the ground-truth mask, the predicted mask, and the overlay between the predicted mask and the input image.

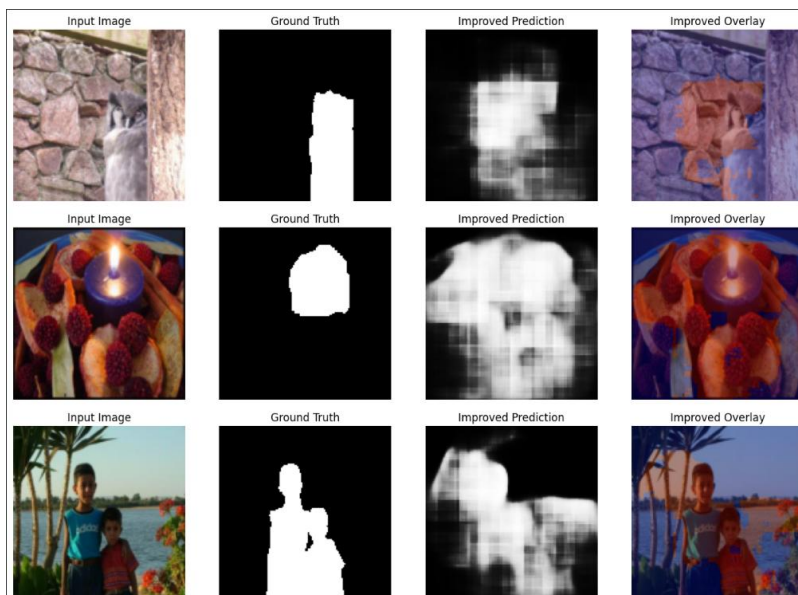


Figure 5: Improved model predictions on test images. The columns show input image, ground-truth mask, improved prediction, and overlay.

8.1 Notebook Demo

The final notebook includes a demo section that loads the best improved checkpoint and allows the user to upload any image. The image is resized to 128x128, converted to a tensor, passed through the model, and displayed as an input image, predicted saliency mask, and overlay. The demo also prints the inference time per image.

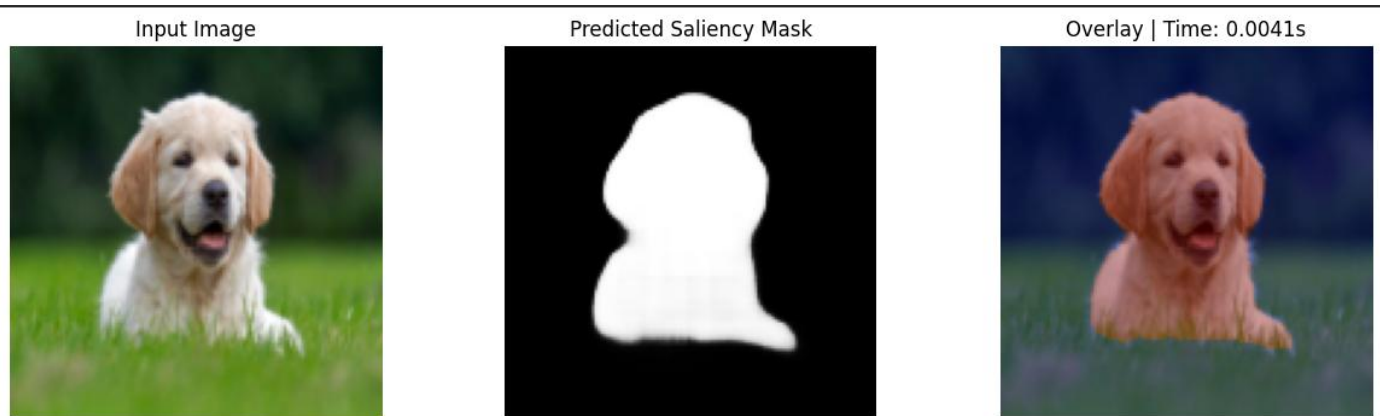


Figure 6: Demo demonstrating the input image, the predicted saliency mask, the overlay between the image and the predicted mask and the inference time per image

9. Discussion, Limitations, and Future Work

9.1 Discussion

The project demonstrates the full deep learning workflow for salient object detection: dataset preparation, preprocessing, augmentation, model design, training, evaluation, comparison, visualization, and demo inference. The improved model achieved better results because it combines stronger feature extraction with regularization and more varied training samples.

9.2 Limitations

- The model uses 128x128 inputs, so fine object boundaries may be lost after resizing.
- The architecture is intentionally simple and does not use skip connections, attention, or pretrained encoders.
- Some predictions can be blurry because the model learns a soft saliency probability map.
- Evaluation is limited to MSRA10K; testing on additional datasets would provide stronger generalization evidence.

9.3 Future Work

- Add U-Net style skip connections to preserve spatial detail.
- Experiment with higher input sizes such as 224x224.
- Add Mean Absolute Error as an additional metric.
- Test the final model on external images and other SOD datasets.
- Create a lightweight Streamlit or Gradio app for easier interaction.

10. Conclusion

This project successfully implements a complete Salient Object Detection system from scratch. The improved CNN encoder-decoder model achieved IoU 0.7270, Precision 0.8068, Recall 0.8851, and F1-score 0.8256, outperforming the baseline model on every metric. The final notebook provides clear visualizations and an upload-image demo with inference time.

10.1 Final Deliverables

- Source code modules: data_loader.py, sod_model.py, train.py, and evaluate.py.
- Colab notebook: demo_notebook.ipynb with training, evaluation, visualizations, and demo.
- Saved checkpoints for baseline and improved models.